

Least Squares Classification with Fisher's Iris Dataset

Lecture Notes

Zamir Hussain
Department of Mathematics
University of Wah

January 9, 2026

Abstract

These comprehensive lecture notes introduce the fundamentals of least squares classification, using Fisher's historic iris dataset as a pedagogical foundation. We explore linear predictor models, decision boundaries, and the critical role of cross-validation in preventing overfitting. Step-by-step mathematical derivations, practical examples, and exercise problems are provided to reinforce understanding.

1 Introduction to Classification & Fisher's Iris Dataset

1.1 What is Classification?

Classification is a supervised learning paradigm where we predict discrete categorical labels based on input features. Unlike regression (which predicts continuous values), classification assigns data points to distinct categories or classes.

Key Concept

Classification Task: Given feature vectors $x_i \in \mathbb{R}^d$ and corresponding discrete labels y_i , learn a mapping $f : \mathbb{R}^d \rightarrow \mathcal{C}$ where $\mathcal{C}$ is a finite set of classes.

1.2 Fisher's Iris Dataset (1936)

This classic dataset contains 150 samples equally distributed among three iris species:

Species	Count	Typical Label Encoding	Distinguishing Features
Setosa	50	+1 or 0	Small petals, well-separated
Versicolor	50	0 or -1	Medium petals
Virginica	50	-1 or +1	Large petals

Four Features (Dimensions):

- (a) Sepal length (cm)
- (b) Sepal width (cm)
- (c) Petal length (cm)
- (d) Petal width (cm)

Mathematical Insight

Mathematical Representation:

- Each flower: $x_i \in \mathbb{R}^4$ (feature vector)
- Each species: $y_i \in \{-1, 0, 1\}$ or alternative encoding
- Complete dataset: $\{(x_i, y_i)\}_{i=1}^{150}$

2 Linear Predictor Model

2.1 Core Mathematical Framework

The fundamental assumption: **Labels are approximately linear combinations of features.**

For a single data point with d features:

$$y_i \approx w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + w_0$$

Component	Description
Weights $w_1, \dots, w_d$	How much each feature contributes to prediction
Offset/Bias w_0	Allows model flexibility (shift)
Features x_{ij}	Measured values (e.g., sepal length = 5.1 cm)

2.2 Matrix Formulation

For n data points with d features, we write compactly:

$$y \approx Xw$$

where:

$$\begin{aligned}
 y &= \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} && \text{(labels vector)} \\
 X &= \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1d} \\ 1 & x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nd} \end{bmatrix}_{n \times (d+1)} && \text{(design matrix)} \\
 w &= \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}_{(d+1) \times 1} && \text{(weights vector)}
 \end{aligned}$$

Key Concept

The column of 1's in X accounts for the offset w_0 , enabling the model to represent decision boundaries that don't pass through the origin.

3 Finding Optimal Weights: Least Squares Solution

3.1 The Optimization Problem

We seek weights minimizing the squared difference between predicted and actual labels:

$$\min_w \|y - Xw\|^2$$

where $\|\cdot\|$ denotes the Euclidean norm: $\|v\| = \sqrt{v_1^2 + \dots + v_n^2}$.

3.2 Closed-Form Solution Derivation

Mathematical Insight

Step-by-Step Derivation:

1. Define error function:

$$E(w) = \|y - Xw\|^2 = (y - Xw)^T(y - Xw)$$

2. Expand:

$$E(w) = y^T y - 2y^T Xw + w^T X^T Xw$$

3. Compute gradient:

$$\nabla E(w) = -2X^T y + 2X^T Xw$$

4. Set gradient to zero:

$$-2X^T y + 2X^T Xw = 0$$

5. Normal equations:

$$X^T Xw = X^T y$$

6. If $X^T X$ is invertible:

$$\hat{w} = (X^T X)^{-1} X^T y$$

Key Concept

The solution $\hat{w} = (X^T X)^{-1} X^T y$ is known as the **ordinary least squares (OLS)** estimator. It exists uniquely when X has full column rank (columns are linearly independent).

3.3 Simple Numerical Example

Example

Consider classifying irises using only sepal length:

Iris	Sepal Length x	Species y
1	5.1 cm	+1 (setosa)
2	7.0 cm	-1 (versicolor)

Step 1: Construct matrices

$$X = \begin{bmatrix} 1 & 5.1 \\ 1 & 7.0 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

Step 2: Compute $X^T X$ and $X^T y$

$$X^T X = \begin{bmatrix} 1 & 1 \\ 5.1 & 7.0 \end{bmatrix} \begin{bmatrix} 1 & 5.1 \\ 1 & 7.0 \end{bmatrix} = \begin{bmatrix} 2 & 12.1 \\ 12.1 & 75.01 \end{bmatrix}$$
$$X^T y = \begin{bmatrix} 1 & 1 \\ 5.1 & 7.0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1.9 \end{bmatrix}$$

Step 3: Calculate inverse

$$\det(X^T X) = 2 \times 75.01 - 12.1 \times 12.1 = 3.61$$

$$(X^T X)^{-1} = \frac{1}{3.61} \begin{bmatrix} 75.01 & -12.1 \\ -12.1 & 2 \end{bmatrix} = \begin{bmatrix} 20.78 & -3.35 \\ -3.35 & 0.554 \end{bmatrix}$$

Step 4: Optimal weights

$$\hat{w} = (X^T X)^{-1} X^T y = \begin{bmatrix} 20.78 & -3.35 \\ -3.35 & 0.554 \end{bmatrix} \begin{bmatrix} 0 \\ -1.9 \end{bmatrix} = \begin{bmatrix} 6.365 \\ -1.0526 \end{bmatrix}$$

Thus: $\hat{y} = 6.365 - 1.0526 \times (\text{sepal length})$

4 Making Predictions: From Continuous to Discrete

4.1 The Sign Function

Linear predictors output continuous values, but classification requires discrete labels.

We apply the **sign function**:

For new observation $\tilde{x}$:

$$\tilde{y} = \text{sign}(\tilde{x}^T \hat{w}) = \begin{cases} +1 & \text{if } \tilde{x}^T \hat{w} > 0 \\ -1 & \text{if } \tilde{x}^T \hat{w} < 0 \\ 0 & \text{if } \tilde{x}^T \hat{w} = 0 \text{ (tie-breaking required)} \end{cases}$$

4.2 Example Continued

Using our model $\hat{y} = 6.365 - 1.0526x$:

- **Test iris with sepal length 6.0:**

$$\tilde{x}^T \hat{w} = 6.365 - 1.0526 \times 6.0 = 0.0494 > 0 \Rightarrow \text{Predicted as Setosa (+1)}$$

- **Test iris with sepal length 6.5:**

$$\tilde{x}^T \hat{w} = 6.365 - 1.0526 \times 6.5 = -0.4769 < 0 \Rightarrow \text{Predicted as Versicolor (-1)}$$

Key Concept

The sign function creates a **hard decision boundary**. All points on one side get the same class label, regardless of distance from the boundary.

5 The Decision Boundary

5.1 Mathematical Definition

The decision boundary is the set where the model is maximally uncertain:

$$\mathcal{B} = \{x \in \mathbb{R}^d : x^T \hat{w} = 0\}$$

Model Type	Boundary Geometry
Without offset ($w_0 = 0$)	Subspace (line/plane through origin)
With offset ($w_0 \neq 0$)	Affine space (shifted line/plane)

5.2 Visualizing with Iris Data

For two features x_1 (sepal length) and x_2 (sepal width):

$$x_1 \hat{w}_1 + x_2 \hat{w}_2 + \hat{w}_3 = 0$$

$$x_2 = -\frac{\hat{w}_3}{\hat{w}_2} - \frac{\hat{w}_1}{\hat{w}_2} x_1$$

5.3 Geometric Interpretation

Mathematical Insight

Normal Vector Interpretation: The weight vector $\hat{w}$ (excluding w_0) is **normal** to the decision boundary. For any two points x_A, x_B on the boundary:

$$\hat{w}^T (x_A - x_B) = 0$$

The offset w_0 determines the boundary's distance from the origin.

6 The Danger of Overfitting

6.1 Training Error vs. Generalization

Critical Insight: Minimizing training error $\|y - \hat{y}\|$ is **not** the ultimate goal!

Example

Overfitting Demonstration: With n data points, a polynomial of degree $n - 1$ can achieve **zero training error** by interpolating all points. However:

- **Training:** Perfect fit (error = 0)
- **Testing:** Poor performance (high error)
- **Reason:** Model memorizes noise rather than learning patterns

6.2 Bias-Variance Tradeoff

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Model Complexity	Bias	Variance
Too simple (underfitting)	High	Low
Just right	Moderate	Moderate
Too complex (overfitting)	Low	High

7 Cross-Validation: Finding the Sweet Spot

7.1 Why Cross-Validation?

To estimate performance on **unseen data** without using the test set during training.

Key Concept

Golden Rule: Never evaluate on the same data used for training!

7.2 Hold-Out Method

1. Split data: Training (70%), Testing (30%)
2. Train on training set
3. Evaluate on testing set

Test Error:

$$\frac{1}{m} \sum_{i \in \text{test}} \mathbb{I}(\hat{y}_i \neq y_i)$$

where $\mathbb{I}$ is indicator function (1 if misclassified, 0 otherwise).

7.3 K-Fold Cross-Validation

1. Partition data into K equal folds
2. For $k = 1$ to K :
 - Train on all folds except fold k
 - Test on fold k
 - Record error e_k
3. Average: CV Error = $\frac{1}{K} \sum_{k=1}^K e_k$

7.4 Leave-One-Out Cross-Validation (LOOCV)

Special case where $K = n$ (number of observations):

- Train on $n - 1$ points
- Test on the 1 left out
- Repeat n times

Advantage	Disadvantage
Maximizes training data Unbiased estimate	Computationally expensive High variance

8 Practical Example: Full Iris Classification

8.1 Binary Classification: Setosa vs. Non-Setosa

Data Preparation:

- 50 Setosa $\rightarrow y = +1$
- 100 others $\rightarrow y = -1$
- Standardize features: $z = \frac{x - \mu}{\sigma}$

Matrix Setup:

$$X = \begin{bmatrix} 1 & 5.1 & 3.5 & 1.4 & 0.2 \\ 1 & 4.9 & 3.0 & 1.4 & 0.2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 5.9 & 3.0 & 5.1 & 1.8 \end{bmatrix}_{150 \times 5}, \quad y = \begin{bmatrix} +1 \\ +1 \\ \vdots \\ -1 \\ -1 \end{bmatrix}_{150 \times 1}$$

8.2 Implementation Steps

1. Compute $\hat{w} = (X^T X)^{-1} X^T y$
2. For new iris: compute $\tilde{x}^T \hat{w}$, apply sign function
3. Evaluate using 10-fold cross-validation

8.3 Typical Results

Metric	Value
Training Accuracy	100%
Test Accuracy (10-fold CV)	98%
Decision Boundary	Linear in 4D space
Misclassified	2-3 flowers (usually Versicolor/Virginica)

9 Summary of Key Concepts

1. **Least Squares Classification:** Minimizes squared error between linear predictions and actual labels
2. **Linear Predictor:** $y \approx Xw$, solved by $\hat{w} = (X^T X)^{-1} X^T y$
3. **Decision Boundary:** $\{x : x^T \hat{w} = 0\}$ separates classes
4. **Sign Function:** Converts continuous outputs to discrete labels
5. **Overfitting:** Complex models memorize training data but generalize poorly
6. **Cross-Validation:** Estimates generalization error using data splitting
7. **Fisher's Iris Dataset:** Classic example with clear linear separability

9.1 Analogy Recap

Example

The Fence Analogy:

- **Linear predictor:** Drawing the best straight fence in a field
- **Weights:** How much to weight different terrain features
- **Cross-validation:** Testing the fence with new animals
- **Overfitting:** Building a twisty fence that fits every existing animal perfectly but confuses new ones

10 Exercise Problems

1. Basic Weight Calculation

Given 3 training points: $(2, +1), (3, +1), (5, -1)$ with model $\hat{y} = w_0 + w_1 x$:

- (a) Set up X matrix and y vector
- (b) Calculate $X^T X$ and $X^T y$
- (c) Solve for $\hat{w} = (X^T X)^{-1} X^T y$
- (d) Find the decision boundary
- (e) Classify new points: $x = 3.0, 3.5, 4.0, 4.5$

2. Decision Boundary Plotting

Using the model from Problem 1:

- (a) Plot the 3 training points
- (b) Draw the decision boundary
- (c) Shade regions for $y = +1$ and $y = -1$
- (d) Mark where a new point at $x = 3.2$ would be classified

3. Overfitting Demonstration

Consider polynomial features up to degree 9 with 10 noisy points from $y = 2x + 3 + \epsilon$:

- (a) Generate 10 points with $\epsilon \sim N(0, 1)$
- (b) Fit linear model ($w_0 + w_1 x$) and 9th-degree polynomial
- (c) Compare training errors
- (d) Predict at $x = 6.5$ with both models

(e) Explain which model generalizes better

4. Cross-Validation Practical

Given 8 points: (1, 2), (2, 3), (3, 5), (4, 4), (5, 6), (6, 5), (7, 8), (8, 9):

- (a) Perform 2-fold CV (Folds: points 1,3,5,7 and 2,4,6,8)
- (b) Calculate test MSE for each fold
- (c) Compute average CV error
- (d) Compare with training error using all 8 points
- (e) Why is CV error more realistic?

5. Iris Dataset Analysis

Using the full iris dataset (available in sklearn or R):

- (a) Perform binary classification (Setosa vs. Non-Setosa)
- (b) Compute confusion matrix
- (c) Calculate precision, recall, F1-score
- (d) Visualize decision boundary for two features
- (e) Implement 5-fold cross-validation

10.1 Sample Solutions

Mathematical Insight

Problem 1 Solution:

(a) $X = \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 5 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$

(b) $X^T X = \begin{bmatrix} 3 & 10 \\ 10 & 38 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

(c) $\det(X^T X) = 14, \quad (X^T X)^{-1} = \frac{1}{14} \begin{bmatrix} 38 & -10 \\ -10 & 3 \end{bmatrix}$
 $\hat{w} = \begin{bmatrix} 4.142 \\ -1.142 \end{bmatrix}$

(d) Decision boundary: $4.142 - 1.142x = 0 \Rightarrow x = 3.626$

(e) Classifications: $x = 3.0, 3.5 \rightarrow +1; \quad x = 4.0, 4.5 \rightarrow -1$

Appendix A: Useful Formulas

Normal Equations $X^T X w = X^T y$

OLS Solution $\hat{w} = (X^T X)^{-1} X^T y$

Decision Boundary $\{x : x^T \hat{w} = 0\}$

Sign Function
$$\text{sign}(a) = \begin{cases} +1 & a > 0 \\ -1 & a < 0 \\ 0 & a = 0 \end{cases}$$

Mean Squared Error
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Accuracy
$$\frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Appendix B: Python Code Snippet

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score

# Load iris dataset
iris = load_iris()
X = iris.data[:100] # First 100 for binary classification
y = iris.target[:100]
y = np.where(y == 0, 1, -1) # Setosa=1, others=-1

# Add intercept term
X = np.hstack([np.ones((100, 1)), X])

# Least squares solution
w = np.linalg.inv(X.T @ X) @ X.T @ y

# Predict function
def predict(x):
    return np.sign(x @ w)

# Cross-validation
scores = cross_val_score(estimator, X, y, cv=5)
print(f"CV Accuracy: {scores.mean():.3f} (±{scores.std():.3f})")
```